

JOB APPLICATION AUTOMATION AGENT — BUILD & DEPLOYMENT PROCEDURE

Project: Personal AI Agent for Resume Tailoring, Application Tracking, Cold Emailing & Semi-Automated Applying **Estimated Duration:** 6-10 days of focused work (can be compressed to 3-4 for an MVP) **Outcome:** A working local/deployed tool that tailors your resume to any JD, tracks every application, sends personalized recruiter emails, and fills job application forms — pausing for your final click before submission.

PROJECT SPECIFICATIONS

Must include: Resume/JD data store, LLM-based tailoring engine, application tracker, email sender, browser automation for form-filling. **Deliverables:** A working local tool (CLI or simple dashboard), GitHub repository, and (optionally) a hosted version.

Customer-facing definition

Build a personal automation agent that takes your resume and a job description as input, and produces: (1) a tailored resume, (2) a personalized cold email to the recruiter, (3) a filled-out job application form ready for your final review and submit click, and (4) a running log of every application you've made.

OVERVIEW

The best way to build this is the same way Datastraw's assessment is structured — as a real, working, end-to-end system, not a proof-of-concept. You already have the technical background to build this in stages: get one thin slice fully working before adding the next.

Duration	Scope	Outcome
6-10 days (or 3-4 for MVP)	Resume tailoring + tracker + email + form-fill agent	Working local tool + GitHub repo

WHAT YOU WILL BUILD

A Python-based agent with four connected modules: 1. **Resume Tailoring Engine** — LLM-powered JD-to-resume matcher and rewriter 2. **Application Tracker** — a database logging every application and its status 3. **Cold Email Generator & Sender** — personalized outreach using your own Gmail 4. **Form-Fill Agent** — browser automation that fills job application forms and stops before submitting

KEY FEATURES (Build in this order)

#	Feature	Details
1	Resume + JD ingestion	Store your resume as structured JSON (not just a PDF); accept a JD as pasted text or URL
2	Tailoring engine	Extract JD keywords, compare against resume JSON, output missing keywords + rewritten bullets
3	Tailored resume export	Generate a clean PDF/DOCX from the tailored JSON for each application
4	Application tracker	Log company, role, JD link, date, resume version used, status (Applied/Interview/Rejected/Offer), follow-up date
5	Cold email generator	LLM drafts a 3-4 line email referencing the JD + your top matching resume points
6	Email sender	Sends via your own Gmail using OAuth, not a bulk mailer
7	Form-fill agent	Opens the job posting, fills standard fields (name, email, phone, resume upload, LinkedIn), stops before submit

TECHNOLOGY STACK (recommended for you)

Given your background (B.Tech IT, comfortable with Python/JS, no frontend experience yet), this stack minimizes frontend work while staying fully functional:

Layer	Choice	Why
Language	Python	Best LLM/automation library support, and likely your strongest language
LLM	Claude API (or free-tier Gemini / local Ollama model)	Structured JSON output, good at rewriting text to match keywords
Database	SQLite (upgrade to Postgres/Supabase later)	Zero setup, file-based, free
Email	Gmail API (OAuth)	Free, sends as you, no bulk-mail red flags
Browser automation	Playwright (Python)	Free, reliable, handles modern JS-heavy forms
Frontend (optional, later)	Plain HTML + a simple template , or skip entirely and run everything as CLI scripts first	You don't need React to get this working — a CLI is enough for personal use
Resume/Doc export	python-docx or a Jinja2 HTML→PDF template	Free, no paid service needed

Deployment (optional)	Railway.app / Render free tier	Only needed if you want it accessible from your phone or want to share it
--------------------------	---	--

You do not need a frontend framework to make this useful — build it as command-line scripts first, add a simple web UI only once the core logic works.

DATABASE DESIGN (keep it simple)

Two tables are enough to start:

APPLICATIONS TABLE - id (pk) - company (text) - role (text) - jd_text (text) - jd_url (text, optional) - resume_version (text — filename or version tag) - status (Draft / Applied / Interview / Rejected / Offer) - applied_at (timestamp) - follow_up_date (date, optional)

CONTACTS TABLE (optional, for cold email) - id (pk) - application_id (fk to applications) - recruiter_name (text) - recruiter_email (text) - email_sent_at (timestamp)

How to actually create this — step by step

1. Install SQLite support (it ships with Python's standard library, no install needed — just `import sqlite3`).
2. Create a file called `db.py` in your project folder.
3. Write a function `init_db()` that connects to a local file `tracker.db` and runs `CREATE TABLE IF NOT EXISTS` statements for both tables above.
4. Run `python db.py` once to create the file — you'll see `tracker.db` appear in your folder.
5. Verify it worked by opening a Python shell, connecting to `tracker.db`, and running `SELECT name FROM sqlite_master WHERE type='table';` — you should see `applications` and `contacts` listed.

```
# db.py
import sqlite3

def init_db():
    conn = sqlite3.connect("tracker.db")
    cur = conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS applications (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            company TEXT, role TEXT, jd_text TEXT, jd_url TEXT,
            resume_version TEXT, status TEXT DEFAULT 'Draft',
            applied_at TEXT, follow_up_date TEXT
        )
    """)
    cur.execute("""
        CREATE TABLE IF NOT EXISTS contacts (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            application_id INTEGER, recruiter_name TEXT,
            recruiter_email TEXT, email_sent_at TEXT,
            FOREIGN KEY (application_id) REFERENCES applications
            (id)
        )
    """)
    conn.commit()
    conn.close()

if __name__ == "__main__":
```

```
init_db()
print("Database initialized.")
```

CORE FUNCTIONS TO BUILD (simple, sufficient)

Build these as plain Python functions first — no need for a REST API unless you later want a web UI:

- `tailor_resume(resume_json, jd_text)` -> `tailored_resume_json`, `missing_keywords`
- `export_resume(tailored_resume_json)` -> `pdf_or_docx_path`
- `log_application(company, role, jd_text, resume_version)` -> `application_id`
- `generate_cold_email(jd_text, resume_json, recruiter_name)` -> `email_draft`
- `send_email(to_address, subject, body)` — via Gmail API
- `fill_application_form(job_url, resume_json)` — opens browser, fills fields, pauses before submit

How to build `tailor_resume()` — step by step

1. **Turn your resume into JSON first.** Open your current resume, and manually copy each section (summary, skills, experience, projects, education) into a structured Python dict, then save it as `resume.json`. This is a one-time task.

```
{
  "name": "Shreyansh Yadav",
  "summary": "...",
  "skills": ["Python", "React", "SQL", "..."],
  "experience": [
    {"title": "...", "company": "...", "bullets": ["...", "..."]}
  ],
  "projects": [{"name": "...", "bullets": ["...", "..."]}],
  "education": [{"degree": "B.Tech IT", "institute": "AKGEC",
    "year": "2022-2026"}]
}
```

2. **Install the Anthropic SDK:** run `pip install anthropic` in your terminal.
3. **Get an API key** from the Anthropic Console and store it as an environment variable (export `ANTHROPIC_API_KEY=your_key_here` on Mac/Linux, or set it in a `.env` file loaded via `python-dotenv`). Never hardcode the key in your script.
4. **Write the prompt carefully** — tell the model exactly what to return (JSON only, specific keys) so you can parse it reliably. Example prompt structure:
 - System instruction: “You are a resume optimization assistant. Given a resume JSON and a job description, return ONLY a JSON object with two keys: `tailored_resume` (same structure as input, with bullets reworded to include relevant JD keywords, and reordered by relevance) and `missing_keywords` (list of important JD terms not present anywhere in the resume).”
 - User message: the JD text + the resume JSON, clearly labeled.
5. **Parse the response** — strip any markdown code fences (`` `json`) before calling `json.loads()`, since models sometimes wrap JSON in them anyway.

6. **Test it** on a real JD — start with the Think41 JD you already have, since you know that role well and can judge if the output makes sense.

```
# tailor.py
import os, json, anthropic

client =
anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

def tailor_resume(resume_json: dict, jd_text: str) -> dict:
    prompt = f"""You are a resume optimization assistant.
    Given this resume JSON and job description, return ONLY a JSON
    object with keys:
    "tailored_resume" (same structure as input, bullets reworded to
    include relevant JD keywords)
    and "missing_keywords" (list of important JD terms not present in
    the resume).

    RESUME JSON:
    {json.dumps(resume_json)}

    JOB DESCRIPTION:
    {jd_text}
    """

    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=2000,
        messages=[{"role": "user", "content": prompt}]
    )
    raw_text = response.content[0].text.strip()
    raw_text = raw_text.replace("`json", "").replace("`",
    "").strip()

    return json.loads(raw_text)
```

How to build export_resume() — step by step

1. Install python-docx: `pip install python-docx`.
2. Write a function that takes the tailored resume JSON and writes each section into a Word document using headings and bullet lists.
3. Save the file with a clear naming convention: `resume_<company>_<role>.docx` so your tracker can reference the exact version used.
4. Optionally convert to PDF afterward using `docx2pdf` (Windows/Mac) or by uploading to Google Docs and exporting, if you're on Linux without a Word-compatible converter.

How to build log_application() — step by step

1. In `db.py`, add a function that opens a connection, runs an `INSERT INTO applications (...)` statement with the values passed in, and returns the new row's id via `cur.lastrowid`.
2. Call this function right after you generate a tailored resume for a JD, so every tailoring attempt is automatically tracked — even before you've applied.
3. Add a second function `update_status(application_id, new_status)` so you can mark applications as "Applied", "Interview", etc. as things progress.
4. Add a `list_applications(status_filter=None)` function that runs a `SELECT * (optionally with a WHERE status = ?)` so you can check your pipeline anytime from a Python shell.

How to build generate_cold_email() — step by step

1. Reuse the same Claude client from `tailor.py`.
2. Write a prompt that includes: the JD text, your resume's top 2-3 most relevant bullets (from the tailoring output), and the recruiter's name if known (otherwise use "Hiring Team").
3. Ask explicitly for a short, specific, non-generic email — 3 to 4 sentences, no buzzwords, referencing one concrete thing from the JD.
4. Always print/preview the draft before sending — treat this the same as the form-fill "pause before submit" principle.

How to build `send_email()` — step by step

1. Go to Google Cloud Console, create a project, and enable the **Gmail API**.
2. Create OAuth 2.0 credentials (Desktop app type) and download the `credentials.json` file into your project folder.
3. Install the Google client libraries: `pip install google-auth-oauthlib google-api-python-client`.
4. On first run, your script will open a browser asking you to log into your own Gmail account and grant "send email" permission — this generates a `token.json` that gets reused after that (no repeated logins).
5. Write `send_email(to_address, subject, body)` using the Gmail API's `users().messages().send()` method with a base64-encoded MIME message.
6. **Test by sending an email to yourself first.** Only point it at a real recruiter address once you've confirmed formatting, subject line, and sender name look correct.

How to build `fill_application_form()` — step by step

(See the dedicated "FORM-FILL AGENT — DETAILED STEPS" section further below — it has the full walkthrough.)

FRONTEND REQUIREMENTS (only once the core works)

Essential pages, if/when you add a UI: - Dashboard listing all applications with status - Form to paste a new JD and trigger tailoring - View of the tailored resume + suggested edits, with an approve/edit step - A "Ready to Apply" queue showing filled-but-not-submitted applications

Keep it plain HTML + minimal CSS (Tailwind via CDN is fine) until the underlying agent works reliably — the UI is not where the value is.

How to add a minimal UI — step by step

1. Install Flask: `pip install flask`.
 2. Create `app.py` with one route / that queries `list_applications()` and renders an HTML table using `render_template`.
 3. Add a second route `/tailor` with a simple form (a `<textarea>` for pasting the JD) that, on submit, calls `tailor_resume()` and displays the result on the page.
 4. Use Tailwind by adding `<script src="https://cdn.tailwindcss.com"></script>` in your HTML `<head>` — no build step needed.
 5. Run locally with `python app.py` and open `http://127.0.0.1:5000` in your browser.
 6. Only move to this step after the CLI/script versions of every function already work — the UI is just a thin display layer on top of functions you've already tested.
-

FORM-FILL AGENT — DETAILED STEPS

This is the trickiest module, so treat it as its own mini-project.

1. Install Playwright:

```
pip install playwright
playwright install chromium
```

2. **Launch a visible browser** so you can see the browser. Never use `headless=True` for this module — you need to see the form before clicking submit yourself.

```
from playwright.sync_api import sync_playwright

def open_job_page(url):
    playwright = sync_playwright().start()
    browser = playwright.chromium.launch(headless=False)
    page = browser.new_page()
    page.goto(url)
    return playwright, browser, page
```

3. **Identify which ATS platform the posting uses** — check the URL. Greenhouse URLs typically contain `boards.greenhouse.io`, Lever URLs contain `jobs.lever.co`. Start by only supporting these two.
4. **Inspect the form's HTML** using your browser's DevTools (right-click → Inspect) on a real posting from that ATS, and note the `id` or `name` attributes of the `name`, `email`, `phone`, `resume-upload`, and `LinkedIn` fields. Save these as a lookup dictionary per platform:

```
SELECTOR_MAPS = {
    "greenhouse": {
        "first_name": "#first_name",
        "last_name": "#last_name",
        "email": "#email",
        "phone": "#phone",
        "resume_upload": "input[type=file]#resume",
    },
    "lever": {
        "full_name": "input[name='name']",
        "email": "input[name='email']",
        "phone": "input[name='phone']",
        "resume_upload": "input[name='resume']",
    }
}
```

5. Fill the standard fields:

```
def fill_standard_fields(page, platform, resume_json):
    sel = SELECTOR_MAPS[platform]
    if platform == "greenhouse":
        page.fill(sel["first_name"], resume_json["name"].split()[0])
        page.fill(sel["last_name"], resume_json["name"].split()[-1])
        page.fill(sel["email"], resume_json["email"])
        page.set_input_files(sel["resume_upload"], resume_json["tailored_resume_path"])
```

6. **Handle free-text questions** (e.g. “Why do you want to work here?”) by locating the `<textarea>` elements on the page, generating an answer via the same Claude client used for cold emails, and filling it in — but keep the text short enough that it's easy for you to skim and edit before submitting.

7. **Take a screenshot for your own review:**

```
page.screenshot(path=f"review_{company}.png")
```

8. **Log the application as “Ready for review”** in your tracker (`update_status(application_id, "Ready for review")`) instead of “Applied” — only change it to “Applied” after you’ve actually clicked submit yourself.

9. **Never write code that calls the submit button’s selector.** This is the single most important rule of this module — leave the browser window open and either wait for a keypress (`input("Press Enter once you've submitted manually...")`) or simply let the script end while the browser tab stays open for you to act on.

10. **Test on a low-stakes posting first** — apply this flow to a role you’re less attached to before using it on one you really want, in case a field gets filled incorrectly.

BONUS / STANDOUT IDEAS (optional)

If you have extra time after the core works: - Add a weekly summary — how many applications sent, response rate, which resume version performs best - Add a “similar past application” check so you don’t accidentally reapply to the same role - Add resume A/B tracking — tag which resume version got interview callbacks

One well-built extra is worth more than several shallow ones — pick the one most useful to your own job search right now.

How to build the weekly summary — step by step

1. Add a function `weekly_summary()` in `db.py` that runs a `SELECT status, COUNT(*)` grouped query over the last 7 days of `applied_at` dates.
 2. Print it as a simple text report, or email it to yourself every Sunday using the same `send_email()` function you already built.
 3. Optionally schedule it with `cron` (Linux/Mac) or Task Scheduler (Windows) so it runs automatically without you remembering to trigger it.
-

CAN YOU USE AI TOOLS?

Yes — this entire project is meant to be built with heavy AI assistance (Claude, Copilot, etc.). Make sure you understand and can explain every function, especially the tailoring prompt logic and the form-filling flow, since you’ll want to debug and extend it yourself.

DEPLOYMENT (only if you want it beyond your own laptop)

Recommended path: - Keep it fully local first — this avoids all cost and most of the ToS risk around browser automation. - If you want a hosted dashboard later: **Railway.app** or **Render** free tier for the backend, **Supabase** free tier if you outgrow SQLite. - Do not deploy the form-fill agent to a public server casually — running Playwright automation from a shared IP on job sites increases the chance of being flagged. Running it from your own machine/IP is safer.

Deployment Note: Don't spend excessive time on hosting. Getting the local tool working reliably is the real goal — hosting is a nice-to-have, not the point.

How to deploy the dashboard (optional) — step by step

1. Push your project to a GitHub repository (see next section for exact commands).
2. Create a free account on [Railway.app](#) or [Render.com](#).
3. Connect your GitHub repo — both platforms auto-detect a Python/Flask app if you include a `requirements.txt` and a `Procfile` (for Render) or just a start command (for Railway).
4. Add your environment variables (`ANTHROPIC_API_KEY`, Gmail credentials) in the platform's dashboard settings — never commit these to GitHub.
5. If you outgrow SQLite (e.g., the file resets on redeploy), create a free Postgres database on [Supabase](#) and swap your `sqlite3` connection for a `psycopg2` connection using the same table schema.
6. **Do not deploy the Playwright form-fill module to the public dashboard** — keep that part running locally on your own machine for the reasons noted above. The hosted dashboard should only show your tracker and tailoring tools.

How to set up the GitHub repository — step by step

1. Run `git init` inside your project folder.
 2. Create a `.gitignore` file and add: `token.json`, `credentials.json`, `.env`, `tracker.db`, `__pycache__`, `/*.pyc` — so secrets and local data never get committed.
 3. Create a `requirements.txt` by running `pip freeze > requirements.txt`.
 4. Write a short `README.md` explaining what the project does, how to set it up locally, and which environment variables are needed.
 5. Run `git add .`, `git commit -m "Initial commit"`, create a new repository on GitHub, then `git remote add origin <repo_url>` and `git push -u origin main`.
-

STEP-BY-STEP BUILD PLAN (day by day)

Day 1-2: Resume tailoring core - Convert your resume into structured JSON - Write `tailor_resume()` using the Claude API - Test against 2-3 real JDs (use your Think41 JD as a first test case) - Export tailored output to PDF/DOCX

Day 3: Application tracker - Set up SQLite with the Applications table - Write `log_application()` and a simple `list_applications()` view (even just printed as a table in terminal)

Day 4: Cold email module - Set up Gmail API OAuth (one-time setup) - Write `generate_cold_email()` and `send_email()` - Test sending to your own email first before using it on a real recruiter

Day 5-6: Form-fill agent - Set up Playwright - Build the selector map for one ATS platform (Greenhouse is a good first target) - Get standard fields filling correctly, then test the “pause before submit” flow end-to-end

Day 7+ (optional): Simple UI + polish - Wrap the four modules behind a minimal dashboard or CLI menu - Add the tracker view and “ready for review” queue

WHAT TO CHECK BEFORE YOU CONSIDER IT “DONE”

- ☐ Can you paste any JD and get a correctly tailored resume back?
 - ☐ Is every application you make logged automatically?
 - ☐ Does the cold email reference specific JD details (not generic filler)?
 - ☐ Does the form-fill agent stop *every time* before submit, with no code path that can auto-submit?
 - ☐ Have you tested the whole flow end-to-end on at least one real job posting?
-

FINAL THOUGHTS

Build it in the order above — resume tailoring first, since that’s useful to you standalone even before the rest exists. Don’t wait until everything is built to start using it on your own applications; test each module on your real job search as you go, starting with roles like the Think41 internship.